

# 第17回 JavaScript の基本 2

## 実習時のフォルダ構成

第 16 回と同じ形式で、Web\_名前 の中に 17 フォルダを作る。17 の中に sample と exercise を作り、次のファイルを用意する。

```
Web_名前
├── 17
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、すべて sample/main.js に書く）
- exercise：演習で作るファイル（演習 1～4、発展）

sample/index.html と exercise/index.html には、どちらも次の内容を書く（第 16 回の sample/index.html をコピーして、<title> だけ変えてもよい）。main.js を読み込んで、ブラウザで実行できる最小限の HTML である。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>JavaScriptの基本 2</title>
</head>
<body>
  <script src="main.js"></script>
</body>
</html>
```

## 講義内容

以降の講義内容のコードは、sample/main.js に、上から順に書き足していく。書いたら sample/index.html をブラウザで開き、F12 キーの開発者ツール（コンソール）で結果を確認する。

### 1. 前回の復習

- console.log(値);：値をコンソールに表示する。
- const：書き換ええない値を入れる変数。let：書き換える値を入れる変数。
- = は「右の値を左の変数に入れる」という意味（代入）。

## 2. データ型

値には「型」（種類）がある。この授業では、次の3つを使う。

| 型             | 例           | 説明          |
|---------------|-------------|-------------|
| 数値 (Number)   | 100 3.14 -5 | 計算に使える      |
| 文字列 (String)  | "勇者" "HP"   | " " で囲む     |
| 真偽値 (Boolean) | true false  | 「はい／いいえ」を表す |

`typeof` を使うと、値の型を調べられる（型を確かめたいときの補助として使う。詳しく覚えなくてよい）。

```
const level = 5;           // 数値
const heroName = "勇者";   // 文字列
const isAlive = true;      // 真偽値
console.log(typeof level); // number
console.log(typeof heroName); // string
console.log(typeof isAlive); // boolean
```

- 数値は `"` で囲まない。5（数値）と `"5"`（文字列）は別のもの。
- 真偽値の `true` と `false` は、小文字で、`"` で囲まずに書く。
- 数値は「計算」、文字列は「文章の組み立て」、真偽値は「比較の結果」に使う。項目3～6で、この使い方を順に学ぶ。

## 3. 算術演算子

計算には、次の記号を使う。

| 記号             | 意味     | 例（結果）                     |
|----------------|--------|---------------------------|
| <code>+</code> | 足し算    | <code>10 + 3</code> → 13  |
| <code>-</code> | 引き算    | <code>10 - 3</code> → 7   |
| <code>*</code> | 掛け算    | <code>10 * 3</code> → 30  |
| <code>/</code> | 割り算    | <code>10 / 4</code> → 2.5 |
| <code>%</code> | 割り算の余り | <code>10 % 3</code> → 1   |

```
console.log(10 + 3); // 13
console.log(10 - 3); // 7
console.log(10 * 3); // 30
console.log(10 / 4); // 2.5
console.log(10 % 3); // 1
console.log(2 + 3 * 4); // 14（掛け算が先）
console.log((2 + 3) * 4); // 20（（ ）の中が先）
```

- 算数と同じで、掛け算・割り算が先に計算される。先に計算したい部分は `( )` で囲む。

変数を使って計算する。ゲームのダメージ計算の例。

```
const attackPower = 40;
const defense = 15;
let enemyHp = 100;

const damage = attackPower - defense;
enemyHp = enemyHp - damage;
console.log(damage, enemyHp);    // 25 75
```

変数の値を計算して、同じ変数に入れ直す書き方には、短い書き方がある。

```
let score = 0;
score += 100;           // score = score + 100 と同じ
score += 50;
console.log(score);     // 150
score -= 30;            // score = score - 30 と同じ
console.log(score);     // 120
```

- `+=`：足して代入する。`-=`：引いて代入する。`*=`：掛けて代入する。

## 4. 文字列の連結とテンプレートリテラル

文字列は `+` でつながられる（連結）。

```
const enemyName = "スライム";
const greeting = enemyName + "が現れた！";
console.log(greeting);           // スライムが現れた！
```

文字列の中に変数の値を入れたいときは、バッククォート ``` で囲み、変数を `${ }` で囲む（テンプレートリテラル）。`+` でつなげるより読みやすい。

```
console.log(`${enemyName}に${damage}のダメージ！`);    // スライムに25のダメージ！
console.log(`残りHP：${enemyHp}`);                     // 残りHP：75
```

- バッククォート ``` は、`"` や `'` とは別の記号。
- `${ }` の中には、変数だけでなく、計算式も書ける。

## 5. 型の違いに注意

`+` は、数値どうしなら足し算、文字列が入ると連結になる。

```
console.log(5 + 3);           // 8
console.log("5" + 3);         // 53（文字列の連結になる）
```

- ここでは、「型が違うと結果が変わる」ことを覚えておけばよい。

## 6. 比較演算子と真偽値

2つの値を比べた結果は、真偽値（`true` か `false`）になる。

| 記号  | 意味    | 例（結果）             |
|-----|-------|-------------------|
| >   | より大きい | 30 > 0 → true     |
| <   | より小さい | 30 < 0 → false    |
| >=  | 以上    | 30 >= 100 → false |
| <=  | 以下    | 30 <= 30 → true   |
| === | 等しい   | 30 === 30 → true  |
| !== | 等しくない | 30 !== 30 → false |

```
const currentHp = 30;
console.log(currentHp > 0);      // true
console.log(currentHp >= 100);   // false
console.log(currentHp === 30);  // true
console.log(currentHp !== 30);  // false
const isDead = currentHp <= 0;
console.log(isDead);            // false
```

- 「等しいか」を比べるときは、`=` を 3 つ並べた `===` を使う。`=` 1 つは代入。
- 比較の結果は、変数に入れられる（`isDead` のように）。

## 演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて、F12 の開発者ツール（コンソール）で確認する。

### 演習 1 データ型を調べる

- 好きなキャラクターの名前、レベル、ボスかどうかを、それぞれ `const` の変数に入れる（変数名の例：`playerName`、`playerLevel`、`isBoss`）。
- 名前は文字列、レベルは数値、ボスかどうかは真偽値（`true` か `false`）にする。
- `typeof` で、それぞれの型を調べて `console.log` で表示する。

### 演習 2 ダメージ計算

- 味方の攻撃力（例：50）と、敵の防御力（例：20）を `const` で用意する。敵の HP（例：100）は `let` で用意する。
- 「攻撃力 - 防御力」でダメージを求め、敵の HP からダメージを引く。これを 2 回行う。
- ダメージと、2 回攻撃した後の敵の HP を `console.log` で表示する。
- 17 個のアイテムを 5 人で同じ数ずつ分けたときの余りを、`%` で求めて表示する。

### 演習 3 文字列を組み立てて表示する

- 敵の名前を `const` の変数に入れる。`+` を使って、「〇〇が現れた！」と表示する。

- テンプレートリテラルを使って、「〇〇に△△のダメージ！残りHPは□□です」と表示する（△△と□□は、演習2の値を使う）。
- `5 + 3` と `"5" + 3` の結果を `console.log` で表示する。結果が違う理由を説明する。

#### 演習4 比較して真偽値を確かめる

- 演習2の続きとして、さらに2回攻撃する。
- 攻撃のたびに、「敵のHPが0より大きいか」（`敵の HP > 0`）の結果を `console.log` で表示する。
- 結果が `true` から `false` に変わることを確認する（HPがマイナスになっても、今回は気にしなくてよい）。

#### 発展（任意）

- 経験値を入れる変数を `let` で作り、0から始める。`+=` を使って、敵を倒すたびに30ずつ、3回増やす。
- そのあと、`*=` を使って、経験値を2倍にする。各段階の値を `console.log` で表示する。